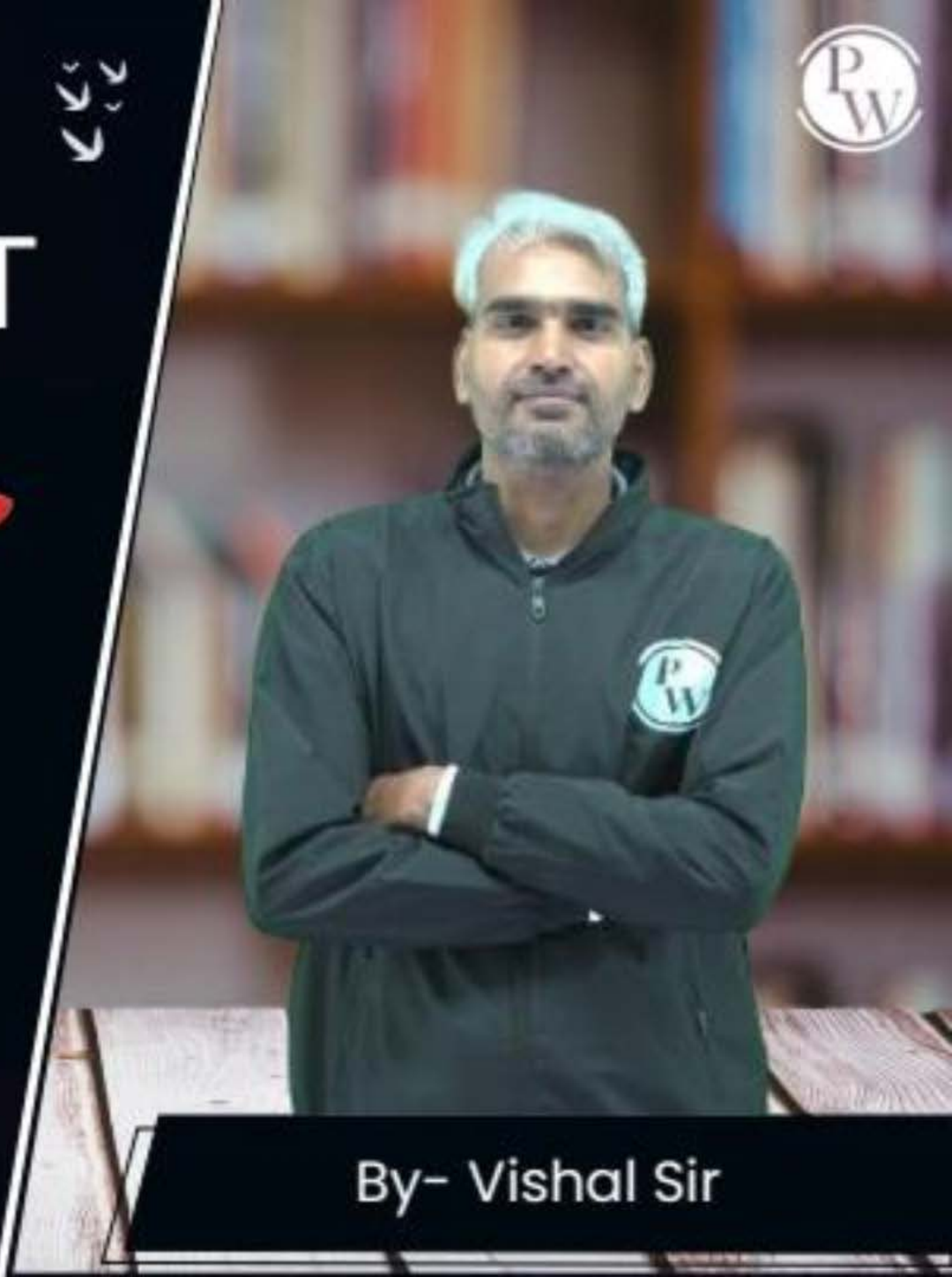
Computer Science & IT



Compiler Design

Intermediate Code Generation
&
Code Optimization

Lecture No. 02



By- Vishal Sir

Recap of Previous Lecture



- **Topic** Three Address Code
- **Topic** Abstract Syntax Tree
- **Topic** Directed Acyclic Graph

Topics to be Covered



- Topic: Code Optimization
- Topic: Static Single Assignment
- Topic: Control Flow Graph

Slide



Topic : Code Optimization

It is a transformation technique such that the resulting code consumes less resources and/or deliver high speed

Slide



Topic : Optimization Techniques

1. Common sub expression elimination
2. Copy propagation
3. Constant folding
4. Dead code elimination
5. Algebraic Simplification
6. Strength Reduction
7. Loop optimization
   - i) code motion
   - ii) Loop merging
   - iii) Loop unrolling



Topic : Common sub expression elimination

An occurance of expression E is called Common Sub-expression if E was computed Previously, and values of variables of expression E have not changed since the Computation of expression E.

Slide

(1)

$$\left\{\begin{array}{l} x = a+b \\ y = 2+a+3+b \\ z = 1+a+b \end{array}\right.$$

Common Sub-expression elimination

$$\begin{array}{l} x = a+b \\ y = 2+3+x \\ z = 1+x \end{array}$$

(2)

$$\begin{array}{l} x = a+b \\ a = c+2 \\ y = 2+a+3+b \end{array}$$

Value of 'a' has Changed
∴ we can not replace a+b by x in statement '3'.



Topic : Copy propagation

Constant Propogation

Variable Propogation

Constant Propagation

$x = 10$ ✓

$y = x + 5$

$Z = y + b$

Constant Propagation ⟹

$x = 10$

$y = 10 + 5$

$Z = y + b$

Variable Propagation

$x = t_1$

$y = x + 5$

$Z = y + b$

Variable Propagation ⟹

$x = t_1$

$y = t_1 + 5$

$Z = y + b$

Slide

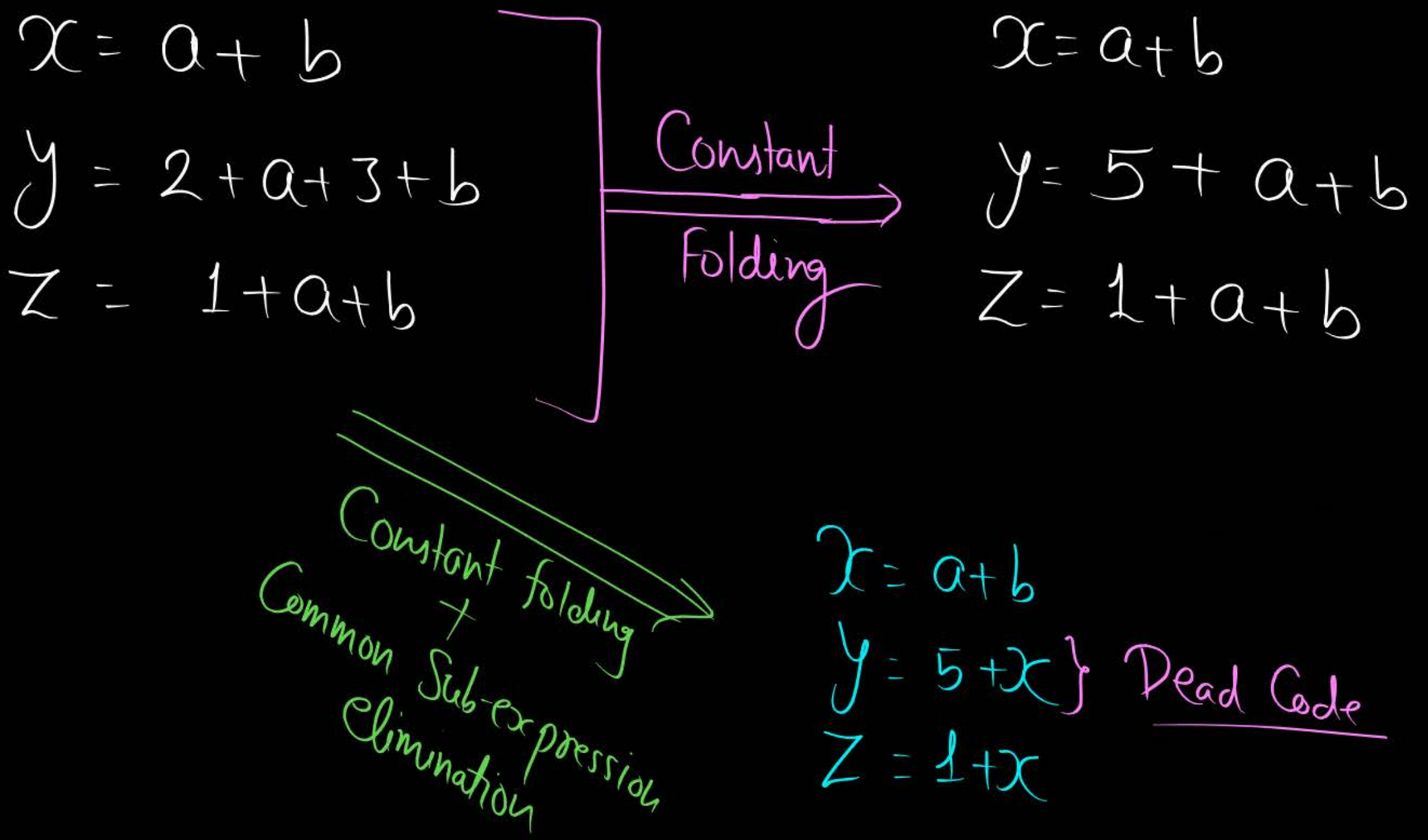
Given a copy statement $x = \underbrace{\text{something}}_{\text{Constant/ variable}}$,

Use that something at place of x whenever Possible after the copy statement



Topic : Constant folding

Based on deducing at Compile time that the Value of an Expression is a Constant, Evaluate that expression and replace the expression by that Constant Value


x = a + b
y = 2 + a + 3 + b
z = 1 + a + b
Constant Folding
x = a + b
y = 5 + a + b
z = 1 + a + b
Constant folding + Common Sub-expression Elimination
x = a + b
y = 5 + x
z = 1 + x
Dead Code



Topic : Dead code elimination

A variable at a point in the program is said to be live if its value can be used subsequently in the program, otherwise it is said to be dead

$x = 10$

$y = 10 + 5$

$z = y + b$

Dead Code elimination

$y = 10 + 5$

$z = y + b$

* Dead Code may be present in the Code because of some other optimizations performed on the original code



Topic : Algebraic Simplification

① $x = a + \cancel{b} - \cancel{b} + c \equiv x = a + c$

② $x = a + \underbrace{b * 1} \equiv x = a + b$

③ $x = a + b + 0 \equiv x = a + b$

④ $x = a + \underbrace{b * 0} \equiv x = a + 0 \equiv x = a$

Slide



Topic : Strength Reduction

* Replace Costly operation with Cheap operation

eg $x = a * 2 \equiv x = a + a$

Costly oph

{ multiplication is Costly whereas addition is Cheap }

eg: $x = a * 2 \equiv x = a << 1$

logical oph

Slide



Topic : code motion

Loop invariant optimization

If an expression inside the loop is loop invariant i.e., its value does not vary with the execution of loop, then such expression can be moved before the loop.

```
i = 0;

while (i ≤ 10000)

{

x = (Sin A * Cos B) * i ;

i = i + 1;

}
```

Value is not going to Change with iterations of the loop.

≡

```
i = 0;
y = Sin A * Cos B;
while ( i ≤ 10,000)
  {
    x = y * i;
    i = i+1;
  }
```

Slide

```
while (i <= (limit-2))
{
    :
    Body
    :
    i=i+1;
}
```

Value of limit does not Change with the iterations of the loop

```
t = limit-2;
while (i <= t)
{
    :
    Body
    :
}
```

Loop Jamming / Loop Fusion

Topic : Loop merging



```
for (i=0; i<10,000; i++)
{
    x = x + i;
}
```

i<10,000 — 10,000 Comparision + increment

x = x + i; Evaluated 10,000 times

```
for (i=0; i<10,000; i++)
{
    y = y * i + 10;
}
```

i<10,000 — 10,000 Comparision + increment

y = y * i + 10; evaluated 10,000 times

≡

Comparision & incrementation time is half

```
for (i=0; i<10,000; i++)
{
    x = x + i;
    y = y * i + 10;
}
```

Slide



Topic : Loop unrolling

```
i =1 ;
while (i ≤ 10000)
{
X[i] = i ;
i++;
}
```

10,000 Comparision

X[i] = i ; } Executed 10,000 times

i++; } 10,000 incrementations

```
i = 1
While (i ≤ 10,000)
{
  x[i] = i ;
  i++ ;
  x[i] = i;
  i++ ;
}
```

No. of Comparision will reduce

only 5000 Comparision

Slide



2 mins Summary

Topic Code Optimization Techniques

Topic Static Single Assignment

Topic Control Flow Graph

THANK - YOU